

Kubernetes - Parte #1

Integração de Sistemas - Aula #08

Prof. Dr. Carlos Melo

 carlos.melo@upe.br

Microserviços no sistema de pedidos

- Na nossa aplicação, pedidos e notificações têm responsabilidades diferentes.
 - Serviço: valida a solicitação e registra o pedido
 - Notificações: consome a mensagem e envia a confirmação
- Uma arquitetura de microserviços organiza a aplicação em serviços com responsabilidades delimitadas e implantação independente
- Podemos publicar uma nova versão de notificações mantendo a versão atual de pedidos, desde que o contrato entre eles permaneça compatível
- O RabbitMQ fornece a infraestrutura de mensageria usada pelos serviços

Confira antes de começar

- Kubernetes administra aplicações em contêineres.
- Vamos experimentá-lo em um cluster local, usando Docker, kind e kubectl.
 - Docker: executa em ambiente local usado pelo kind
 - kind: cria um cluster Kubernetes na nossa máquina
 - kubectl: envia comandos para o cluster
- Verifique:
 - `docker ps`
 - `kind version`
 - `kubectl version --client`

Kubernetes na nossa máquina

- Um *cluster* reúne os nós administrados pelo Kubernetes
- Cada nó oferece recursos para executar as aplicações
- Crie o ambiente da aula:
 - `kind create cluster --name integracao --wait 120s`
 - `kubecttl config use-context kind-integracao`
 - `kubecttl get nodes`
- Ao concluir, procure um nó chamado `integracao-control-plane` com o estado `Ready`.

Nginx no Kubernetes

- Vamos executar a mesma imagem usada na aula de Docker:
 - `kubectl create deployment site --image=nginx:alpine`
 - `kubectl rollout status deployment/site --timeout=120s`
 - `kubectl get pods`
- O *Deployment* declara como a aplicação deve executar e gerencia suas réplicas.
- Esse comando solicita uma réplica.
- O contêiner do Nginx executa em um Pod, a unidade que o Kubernetes agenda em um nó.
- Na listagem, procure um Pod cujo nome começa com `site-`, com `READY` igual a `1/1` e `STATUS` igual a `Running`.

A página no navegador

- Abra um encaminhamento temporário até a aplicação:
 - `kubectl port-forward deployment/site 8080:80`
- Acesse `http://localhost:8080`.
 - A página deve mostrar `Welcome to nginx!`
 - Mantenha o terminal aberto
 - Use outro terminal para os próximos comandos.
- `Ctrl+C` encerra o encaminhamento
- A aplicação continua executando no cluster.

Remoção de um Pod

- No terminal do port-forward, pressione Ctrl+C.
- Em seguida, acompanhe os Pods:
 - `kubectyl get pods -l app=site -w`
 - `-l app=site` seleciona os Pods da aplicação. `-w` acompanha as mudanças.
- Em outro terminal, remova o Pod da aplicação:
- Neste momento, temos apenas uma réplica.
- Observe o Pod antigo sair e outro, com nome diferente, chegar a Running.
- O Kubernetes criou um novo Pod para repor o que foi removido.
- Ao terminar a observação, pressione Ctrl+C no terminal que acompanha os Pods.

Estado desejado e estado atual

- O Deployment continua configurado para manter uma réplica, mesmo depois da remoção do Pod.
- O Kubernetes compara a quantidade desejada com os Pods existentes e ajusta essa quantidade. É só checar:
 - `kubectl get deployment site`
- Depois da reposição, READY deve mostrar 1/1: uma réplica pronta de uma desejada.

Três réplicas da mesma aplicação

- Altere a quantidade desejada:
 - `kubectl scale deployment site --replicas=3`
 - `kubectl rollout status deployment/site --timeout=120s`
 - `kubectl get pods -l app=site -o wide`
- Ao concluir, devem aparecer três Pods em Running, cada um com READY igual a 1/1.
- São três instâncias da mesma aplicação.
 - Todos executam nginx:alpine
 - Todos os nós estão no único nó disponível
 - Cada Pod possui seu nome e IP.

Um endereço para as réplicas

- O Service fornece um nome e um IP internos estáveis para acessar os Pods da aplicação.
 - `kubectl expose deployment site --port=80 --target-port=80`
 - `kubectl get service site`
- O comando cria o Service site e usa o rótulo `app=site` para selecionar os Pods.
 - `--port=80` define a porta pela qual o Service recebe conexões
 - `--target-port=80` define a porta da aplicação nos Pods selecionados
- O tipo padrão é ClusterIP, com acesso interno ao cluster.
- Novas conexões podem ser encaminhadas para diferentes Pods prontos.

Acesso pelo nome do Service

- Vamos fazer uma requisição de dentro de um dos Pods:
 - `kubectl exec deployment/site -- wget -qO- http://site`
 - `exec deployment/site` executa um comando no contêiner de um Pod desse *Deployment*
 - `--` separa as opções do `kubectl` do comando executado
 - `wget -qO- http://site` consulta o endereço e escreve o corpo da resposta no terminal.
- A resposta deve conter o HTML com Welcome to nginx!
 - O nome `site` identifica o Service dentro do mesmo namespace
 - O Kubernetes resolve esse nome e encaminha a conexão a um dos Pods

Qual réplica respondeu?

- Todos os Pods devolvem a mesma página.
- Vamos identificar quem atendeu pelos *logs*.
- Em um terminal, acompanhe os registros:
 - `kubectl logs -l app=site --prefix -f --tail=0`
 - `--prefix` mostra o nome do Pod e do contêiner em cada linha.
 - `--tail=0` inicia o acompanhamento sem imprimir registros antigos.
- Em outro terminal, faça uma requisição:
 - `kubectl exec deployment/site -- wget -qO- http://site`
- No primeiro terminal, localize a linha com GET / e o status 200.
- O prefixo identifica o Pod que respondeu.

Requisições entre réplicas

- Mantenha os logs abertos.
- No outro terminal, entre em um Pod:
 - `kubectl exec -it deployment/site -- sh`
- Dentro dele, envie dez requisições:



```
for i in 1 2 3 4 5 6 7 8 9 10
do
  wget -qO /dev/null http://site
done
```

- `/dev/null` descarta o HTML
- Acompanhe as requisições pelos logs e compare os nomes dos Pods que responderam
- Cada execução de `wget` abre uma nova conexão
- A quantidade recebida por cada réplica pode ser diferente
- Use `exit` para sair do contêiner e `Ctrl+C` para encerrar os logs.

Deployment em YAML

- Vamos registrar a configuração da aplicação em deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: site
spec:
  replicas: 3
  selector:
    matchLabels:
      app: site
  template:
    metadata:
      labels:
        app: site
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
```

- replicas define a quantidade desejada.
- selector identifica os Pods gerenciados.
- template descreve os Pods que serão criados, incluindo seu rótulo e sua imagem.

Service em YAML

- Na mesma pasta, crie `service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: site
spec:
  type: ClusterIP
  selector:
    app: site
  ports:
    - port: 80
      targetPort: 80
```

- O seletor `app: site` conecta esse Service aos Pods definidos no *Deployment*.
- Os dois arquivos representam a configuração que já experimentamos pelo terminal:
 - Três réplicas do Nginx e
 - Um endereço interno para acessá-las.

Recriação pelos arquivos

- Na pasta dos dois arquivos YAML, remova os recursos site do laboratório e recrie-os:
 - `kubectl delete deployment/site service/site`
 - `kubectl apply -f deployment.yaml -f service.yaml`
 - `kubectl rollout status deployment/site --timeout=120s`
- Confira os Pods e faça outra requisição:
 - `kubectl get pods -l app=site`
 - `kubectl exec deployment/site -- wget -qO- http://site`
- O resultado esperado é a aplicação com três Pods prontos e a página do Nginx acessível pelo nome site.
- Agora podemos guardar os arquivos no Git e usar `kubectl apply` para criar ou atualizar os recursos descritos neles.

Nossa página no Kubernetes

- Vamos retomar o Dockerfile da aula de Docker.
- Na pasta dos YAML, crie index.html
- Crie também o Dockerfile:

```
<!doctype html>
<meta charset="utf-8">
<title>Aplicação da turma</title>
<h1>Aplicação da turma</h1>
<p>Versão 1.0</p>
```

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
```

- Construa a imagem nessa pasta:
 - `docker build -t meu-site:1.0 .`
- O COPY inclui a página na imagem
- A tag 1.0 identifica esta versão

Usando a imagem no cluster

- Carregue a imagem do Docker nos nós do kind:
 - `kind load docker-image meu-site:1.0 --name integracao`
- Em `deployment.yaml`, ajuste o contêiner em `spec.template.spec.containers`:



```
containers:
  - name: nginx
    image: meu-site:1.0
    imagePullPolicy: IfNotPresent
```

- `IfNotPresent` permite usar a imagem já carregada no nó.
 - `kubectl apply -f deployment.yaml`
 - `kubectl rollout status deployment/site --timeout=120s`
 - `kubectl exec deployment/site -- wget -qO- http://site`
- Resultado esperado:
 - HTML contendo `<p>Versão 1.0</p>`

Preparando uma nova versão

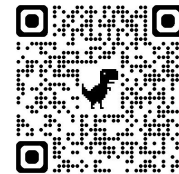
- Em `index.html`, troque o parágrafo por `<p>Versão 2.0</p>`
- Construa uma nova imagem e carregue-a no kind:
 - `docker build -t meu-site:2.0 .`
 - `kind load docker-image meu-site:2.0 --name integracao`
- A aplicação ainda está na versão 1.0.
- Carregar uma imagem não altera o *Deployment*.
- Antes de publicar a mudança, abra outro terminal e acompanhe os Pods:
 - `kubectrl get pods -l app=site -w`
 - Deixe esse terminal aberto para observar a próxima etapa.

Atualização gradual dos Pods

- Em `deployment.yaml`, altere a imagem: `image: meu-site:2.0`
- No terminal livre, aplique a mudança:
 - `kubectl apply -f deployment.yaml`
 - `kubectl rollout status deployment/site --timeout=120s`
- Observe os novos Pods surgindo e os antigos sendo encerrados
- Essa substituição é chamada de `rolling update`, a estratégia padrão do Deployment
- Depois que a atualização terminar:
 - `kubectl exec deployment/site -- wget -qO- http://site`
- Resultado esperado: `<p>Versão 2.0</p>`

Voltando à versão anterior

- Vamos simular que a versão 2.0 precisa ser retirada.
- Reverta a última atualização:
 - `kubectl rollout undo deployment/site`
 - `kubectl rollout status deployment/site --timeout=120s`
 - `kubectl exec deployment/site -- wget -qO- http://site`
- O HTML volta a mostrar <p>Versão 1.0</p>.
- Para registrar a decisão no arquivo, volte a imagem em deployment.yaml para:
 - `image: meu-site:1.0`
- Reaplique o arquivo:
 - `kubectl apply -f deployment.yaml`
- Encerre o acompanhamento dos Pods com Ctrl+C.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br